

Generative AI models for Muography

Author

Shaivya Shankar
hv24220@bristol.ac.uk

Supervised by:

Sudarshan Paramesvaran
sudarshan.paramesvaran@cern.ch

Florian Bury
florian.bury@cern.ch

Abstract

Generative AI represents a significant shift in artificial intelligence, enabling the creation of new data rather than merely analyzing or classifying existing inputs. This capability is particularly valuable in scientific fields where traditional simulations are computationally demanding. In muography, for instance, simulating muon propagation is resource-intensive and costly. To address this challenge, this project investigates generative adversarial networks (GANs) as complementary tools to conventional muon-propagation simulators. The work began with an implementation of the original 2014 vanilla GAN, followed by an evaluation of normalization strategies to improve stability during training. Building on these insights, Wasserstein GANs (WGANs) were implemented to further enhance convergence and mitigate mode collapse. Finally, a conditional GAN was explored to assess how GANs can simulate muon propagation across different materials, leveraging conditioning to improve both control and accuracy of the generated results. Each model was systematically benchmarked in terms of stability, efficiency, and accuracy, with comparisons highlighting their respective strengths and limitations. The findings suggest that original GAN-based approaches can complement conventional simulation frameworks by offering scalable, computationally efficient surrogates.

0.1 Ethics Statement

I have discussed the ethics of this project with my supervisors, and no formal ethics approval is required as the research uses simulated data and does not involve human or animal subjects.

Contents

0.1	Ethics Statement	1
1	Introduction	4
2	What is Generative Adversarial Networks	4
2.1	Vanila GAN 2014	4
2.2	Mathematical Formulation of Objective Function	5
2.2.1	GAN Training Loop	5
2.3	Issues with Vanilla GANs	6
2.4	Methods For stablizing the GAN	6
2.4.1	Normalization as a Stabilization Technique	7
2.5	Wasserstein GAN (WGAN) 2017	9
2.5.1	The Lipschitz Constraint in Wasserstein GANs	10
2.6	Training Loop for the Wasserstein GAN	11
2.7	Conditional Generative Adversarial Networks (cGANs)	11
2.8	Training Loop for the Conditional GAN	12
3	Data Aquisition	12
3.1	Data Normalization and Preprocessing	13
3.2	Visual Validation of the Normalization Process	13
3.3	Implementation Framework	13
3.4	Results and Selection	14
4	Code Implementation	15
4.1	Modular GAN Class Design.	15
4.1.1	Optimal Discriminator Configuration.	15
4.2	The <code>GANDataset</code> Class: Data Preparation for Adversarial Training	16
4.2.1	Inheritance and Core Functionality	16
4.2.2	Data Partitioning and Normalization	17
4.3	The <code>GANSamplingCallback</code> Class: Evaluating Generator Performance	17
4.3.1	Inheritance and Reusability	17
4.3.2	The <code>process</code> Method: Generating Statistical Samples	18
4.3.3	The <code>plot_sampling</code> Method: Visualizing the Output	18
4.4	The <code>GANSamplingCallback</code> Class: Evaluating Generator Performance	19
4.4.1	Inheritance and Reusability	19
4.5	The <code>GANSamplingCallback</code> Class: Evaluating Generator Performance	19
4.5.1	The <code>BaseSamplingCallback</code> : A Reusable Plotting Engine	19
4.5.2	Inheritance and the GAN-Specific Implementation	19
4.5.3	The <code>process</code> Method: Generating Statistical Samples	19
4.5.4	The <code>plot_sampling</code> Method: Visualizing the Output	20
4.6	Visualization of 2D Data Correlations of Orignal Data	20
4.6.1	Functionality and Design	20
4.6.2	Output and Interpretation	21
4.7	The <code>plot_distributions</code> Function: Visualizing Data Transformations	21
4.7.1	Functionality and Design	21
4.8	Hyperparameter Optimization with Optuna	21
4.8.1	The Objective Function	21
4.8.2	Pruning Strategies	22
4.8.3	Study Execution	22
4.9	Code Implementation of Normalization	22
4.9.1	Code Implementation of the Wasserstein GAN	23
4.9.2	Code Implementation of the Wasserstein GAN	23
5	Results and Discussion	23
5.1	Optuna study based Hyperparameters	23
5.2	Vanila 2014 GAN Results	24
5.3	Venial Gan with Normalization	26
5.4	Vanilla GAN with Normalization	26
5.5	WGAN: Spectral Normalization vs. Weight Clipping	27
5.6	Conditional GAN: Element-wise Comparison	28

6

Conclusion and Future Work

6.1

Conclusion

29

6.2

Future Work: Full Trajectory Validation

29

1 Introduction

Generative Artificial Intelligence (AI) represents a paradigm shift in scientific computing, offering powerful new methods for modeling complex systems. These models learn the underlying probability distributions within large datasets, enabling them to generate synthetic data whose probability distribution closely matches that of the original training data. This capability is particularly valuable in physics, where such models can provide high-speed, complementary alternatives to traditional but computationally expensive simulation techniques. This project applies generative modeling to the field of **muography**, a non-invasive imaging technique that utilizes high-energy cosmic-ray muons to probe the interior of large, dense structures. The current gold standard for simulating muon propagation is based on Monte Carlo methods, such as those implemented in Geant4. While highly accurate, these simulations are resource-intensive because a muon’s trajectory through matter is inherently stochastic and governed by complex physical interactions. GANs are not intended to replace these conventional simulators, but rather to complement them by providing computationally efficient surrogates that can accelerate exploratory and large-scale studies. Among generative models, **Generative Adversarial Networks (GANs)** have emerged as promising candidates. GANs are implicit generative models, learning to directly sample from complex data distributions without requiring an explicit probability density function. In this work, the GAN framework is applied to muon propagation by taking as inputs the initial momentum and radiation length (X_0) of a material segment and generating the corresponding scattering angle $\Delta\theta$, Step Length Δr , Momentum Loss Δp . As a first step, key research questions were posed: Which GAN architectures are most effective for modeling muon propagation? How does normalization affect the stability of MLP-based GANs in physics contexts? Can conditional GANs capture the behavior of muons in different materials? The investigation proceeded in stages. First, a vanilla GAN was implemented, and its architecture was optimized using Optuna to identify stable configurations. To further improve stability, normalization strategies—batch normalization, spectral normalization, and layer normalization—were systematically evaluated; all of them showed stabilizing effects on the GAN but they were no better than the GAN without any normalization. After trying methods to stabilize the GAN, Wasserstein GAN with Weight Clipping and spectral Normalization was implemented, both showed convergence similar to that of original GAN while requiring relatively high hyperparameter tuning. Finally, a conditional GAN was developed, based on the most stable vanilla GAN architecture. This conditional model demonstrated the best overall performance, effectively simulating muon propagation across different materials while maintaining stability and accuracy. The overall goal of this project is to assess the suitability of GAN-based models as efficient complements to Monte Carlo simulations in muography. The findings provide insights into architecture stability, normalization effects, and the role of conditional generative models in capturing multi-material muon propagation.

2 What is Generative Adversarial Networks

Generative Adversarial Networks (GANs) are a class of machine learning frameworks designed to learn complex data distributions through an adversarial setup between two competing neural networks. This adversarial nature makes GANs particularly powerful for generating new data samples that closely resemble real data, as the networks continuously improve in opposition to one another during training. The framework consists of two key components: a generator network that takes random noise as input and transforms it into synthetic samples, and a discriminator network that distinguishes between real data from the training set and fake data produced by the generator. The generator’s primary objective is to produce samples whose distribution $p_g(x)$ becomes indistinguishable from the true data distribution $p_{\text{data}}(x)$, while the discriminator aims to correctly classify samples as real or generated. This interaction is formalized as a min-max optimization problem, where the discriminator maximizes its ability to classify samples correctly, while the generator minimizes its loss by reducing the discriminator’s ability to differentiate between real and synthetic data. Through this adversarial training process, both networks improve iteratively—the discriminator becomes better at detecting imperfections, and the generator learns to eliminate them. The training dynamics are driven by this min-max objective, which encourages realistic sample synthesis. Each GAN variant addresses the core adversarial training challenge differently, but all share the fundamental principle of iterative improvement through competition. The training dynamics encourage realistic sample synthesis as the generator learns to eliminate imperfections that the discriminator identifies.

2.1 Vanila GAN 2014

A vanilla GAN is the original 2014 setup with two neural networks trained in opposition. The generator takes random noise and produces synthetic samples; the discriminator evaluates inputs and predicts whether they are real or fake. Training is framed as a two-player game: the discriminator learns to correctly identify real versus generated data, while the generator learns to fool the discriminator. At the ideal balance point, the difference between the real-data distribution and the generator’s distribution—measured by the Jensen-Shannon divergence—has been driven as low as possible, meaning generated samples are effectively indistinguishable from real ones.

2.2 Mathematical Formulation of Objective Function

The GAN framework implements a two-player minimax game with the following objective function, as originally formulated by Goodfellow et al. (2014):

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (1)$$

Where:

- G is the Generator network
- D is the Discriminator network
- x represents real data samples from the true distribution $p_{\text{data}}(x)$
- z is random noise sampled from a prior distribution $p_z(z)$
- $G(z)$ represents generated samples
- $D(x)$ outputs the probability that x is real or fake

Task of Generator

- The fundamental objective is the same as in a traditional GAN: to process a random noise vector and conditional inputs (P, X_0) to synthesize realistic three-dimensional physics outputs ($-\Delta P, \Delta\theta, \Delta r$).
- Instead of trying to fool a binary classifier, the generator’s goal is to produce samples that earn a high “realness” score from the **critic**.
- By maximizing this score, the generator implicitly minimizes the **Wasserstein distance** between the real and generated data distributions.
- This process provides a smoother and more stable gradient, which significantly mitigates the vanishing gradient problem and helps prevent mode collapse.

Task of Critic

- The Critic replaces the traditional discriminator and its task is to output an unbounded score that estimates the Wasserstein distance, rather than a probability.
- It must adhere to a **1-Lipschitz continuity** constraint to function correctly.
- This constraint is enforced via the hyperparameter **weight clipping** (c): a small value of c can limit the critic’s learning capacity, while a large value can lead to instability.
- The critic is updated more frequently than the generator, controlled by the hyperparameter n (e.g., $n = 5$ critic updates per generator update), to ensure it provides an accurate gradient.
- Its architecture is typically an MLP with a final linear activation function to allow for the unconstrained score output.

2.2.1 GAN Training Loop

The training of a GAN is a zero-sum game where one network’s gain is the other’s loss, in this project training loop is similar as the original GAN 2014 paper. The process unfolds in an alternating fashion:

Step 1: Training the Discriminator

First, the Discriminator is trained for one or more epochs. During this phase, the Generator’s weights are frozen.

- **Real Data Batch:** A batch of real data samples is selected from the training dataset. These are fed into the Discriminator, which is trained to output a high probability (close to 1), indicating that they are real.
- **Fake Data Batch:** The Generator takes a batch of random noise vectors and produces a batch of fake data samples.
- **Discriminator Evaluation:** These fake samples are then fed to the Discriminator. The Discriminator is trained to output a low probability (close to 0) for these fake samples.
- **Backpropagation:** The Discriminator’s weights are updated based on the errors it made in classifying both the real and fake data. This process hones its ability to differentiate between authentic and synthetic data.

Step 2: Training the Generator

Next, the Generator is trained for one or more epochs with the Discriminator’s weights held constant.

- **Generate Fake Data:** The Generator creates a new batch of fake data from random noise.
- **Fooling the Discriminator:** This fake data is passed through the (now frozen) Discriminator.
- **Generator’s Loss Calculation:** The Generator’s loss is calculated based on how well it fooled the Discriminator. Specifically, the Generator aims to produce outputs that the Discriminator classifies with a high probability (close to 1).
- **Backpropagation:** The error is backpropagated through the Discriminator (without updating its weights) to the Generator. This provides the Generator with a gradient signal indicating how it should adjust its weights to produce more realistic data.

Step 3: Iteration

These two steps are repeated iteratively. As the Generator becomes better at creating realistic data, the Discriminator’s job becomes more challenging. Conversely, as the Discriminator improves at identifying fakes, the Generator must learn to produce even more convincing forgeries. **Optimization Details:** The gradient-based updates can utilize any standard gradient-based learning rule.

2.3 Issues with Vanilla GANs

Although Generative Adversarial Networks (GANs) introduced by Goodfellow et al. [4] were a breakthrough in generative modeling, the original (vanilla) formulation suffers from several well-known drawbacks:

1. **Training Instability:** The adversarial min-max optimization often leads to oscillations rather than convergence, making training highly unstable [15, 12].
2. **Mode Collapse:** The generator may produce only a limited variety of outputs (e.g., generating samples from only a few modes of the true distribution), ignoring the full diversity of the dataset [1, 17].
3. **Vanishing Gradients:** When the discriminator becomes too strong, the generator receives almost no gradient signal (due to saturating loss functions), which stalls learning [4, 15].
4. **Lack of a Meaningful Loss Metric:** The vanilla GAN loss does not correlate reliably with sample quality, making it difficult to evaluate progress during training [1, 9].
5. **Hyperparameter Sensitivity:** Successful training requires delicate balancing of generator and discriminator architectures, learning rates, and update frequencies [17, 11].
6. **Incomplete Data Coverage:** Even when producing realistic samples, the generator may fail to capture the full distribution, resulting in mode dropping [1, 12].

2.4 Methods For stabilizing the GAN

Though GAN is unstable there are ways to stabilize the training of traditional GAN, To overcome these issues and make GAN training more robust, researchers have developed a variety of powerful stabilization techniques. These stabilization methods can be broadly grouped into three main categories, each tackling the problem from a different angle:

1. **Modifying the Objective Function:** This approach replaces the original binary cross-entropy loss, which implicitly minimizes the Jensen-Shannon Divergence, with a more stable alternative. By using loss functions with better gradient properties, such as the **Wasserstein distance** in WGANs or the generator receives a more reliable signal for improvement throughout the training process.
2. **Applying Normalization and Regularization:** These techniques are applied directly to the discriminator to control its behavior and prevent it from overpowering the generator. **Spectral Normalization**, for example, enforces a Lipschitz constraint on the discriminator’s layers, which helps control the magnitude of its gradients. Similarly, **Batch Normalization** helps regulate the flow of information and stabilize learning, though it must be used carefully to avoid introducing artifacts.
3. **Refining the Network Architecture:** This strategy involves making specific design choices that are inherently more stable. The **Deep Convolutional GAN (DCGAN)** architecture provided an early blueprint for this, establishing rules like replacing pooling layers with strided convolutions. More advanced techniques, like the **Progressive Growing** of networks, offer further stability by training the model on progressively higher-resolution images.

By strategically combining these approaches—a better loss function, a well-regulated discriminator, and a stable architecture—it is possible to transform the notoriously difficult training process of a traditional GAN into a more reliable and effective framework for generative modeling.

2.4.1 Normalization as a Stabilization Technique

In the context of deep neural networks, normalization refers to a regularization technique that standardizes the input distributions to individual layers, maintaining activation magnitudes within reasonable bounds throughout the network. Research has demonstrated that this technique prevents numerical instabilities that arise from excessively large or vanishing activation values during forward propagation. Despite its computational simplicity, empirical evidence suggests that normalization has profound effects on optimization dynamics by addressing fundamental challenges in gradient-based learning. The primary benefit of normalization lies in its ability to improve the optimization landscape. It has been observed that without normalization, activation values can vary dramatically across network layers, forcing subsequent layers to constantly adapt to these changes. This phenomenon is known as internal covariate shift, where the statistical distribution of layer inputs continuously changes as parameters in earlier layers are updated during training. Theoretical analysis suggests that by maintaining stable activation statistics, normalization allows layers to learn more independently, enables higher learning rates, and promotes faster convergence. These stabilization properties have established normalization as an important technique for addressing the training instabilities commonly found in adversarial learning. The underlying principle is that explicit control of activation statistics in both generator and discriminator networks can prevent the problematic behaviors that often occur in adversarial optimization. This study systematically evaluates three established normalization techniques to assess their effectiveness in stabilizing GAN training and improving generative model performance.

- **Batch Normalization (BN):** Introduced by Ioffe and Szegedy in 2015 [5], Batch Normalization (BN) emerged as an important solution to several persistent challenges in training deep neural networks. Prior to its development, researchers faced significant difficulties with internal covariate shift, gradient instabilities, and convergence issues that limited the practical depth of neural architectures.

BN addresses these problems by standardizing the activations within each layer through a systematic approach. The technique calculates the mean and variance across all samples in the current mini-batch, then uses these statistics to normalize the activations, centering them around zero with unit variance. To preserve the network’s learning capacity, BN incorporates two learnable parameters that can scale and shift these normalized values, allowing the network to adapt the distribution as needed during training.

The introduction of BN brought several practical improvements to neural network training. It accelerated convergence rates, reduced dependency on careful weight initialization, enabled the use of higher learning rates, and made it more feasible to train deeper architectures. These improvements contributed to the development of more complex models that have become common in modern deep learning applications.

$$\begin{aligned}\mu_{\mathcal{B}} &= \frac{1}{|\mathcal{B}|} \sum_{x_i \in \mathcal{B}} x_i, & \sigma_{\mathcal{B}}^2 &= \frac{1}{|\mathcal{B}|} \sum_{x_i \in \mathcal{B}} (x_i - \mu_{\mathcal{B}})^2 \\ \tilde{x}_i &= \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, & y_i &= \gamma \odot \tilde{x}_i + \beta\end{aligned}$$

The batch normalization process follows these sequential steps:

- $\mu_{\mathcal{B}}, \sigma_{\mathcal{B}}^2$: First, BN computes the empirical mean and variance across all samples x_i in the current mini-batch $\mathcal{B}$. These statistics capture the distribution of activations for the entire batch at this layer.
- $\tilde{x}_i$: Next, each activation x_i is normalized by subtracting the batch mean and dividing by the batch standard deviation. This transformation centers the data around zero and scales it to unit variance. The small constant ϵ prevents numerical instability when the variance is very small.
- y_i : Finally, the normalized activations are transformed using two learnable parameters: γ scales the normalized values while β shifts them. This step allows the network to learn the optimal scale and location for the activations, potentially recovering the original distribution if that proves beneficial for the task.

Various empirical studies have shown that BN performs particularly well in generator networks, especially for image synthesis tasks with sufficient batch sizes. The technique has been demonstrated to stabilize activation distributions, improve optimization properties, and accelerate convergence. However, research suggests that applying BN in discriminator networks can introduce dependencies between samples within the same batch. It has been observed that discriminators may exploit these batch-level correlations rather than learning individual sample features, potentially disrupting the adversarial training process. Hence Batch normalization is applied on generator only

- **Layer Normalization (LN):** Layer Normalization, introduced by Ba et al. in 2016 [2], was developed to address several limitations of Batch Normalization, particularly its dependence on batch statistics and poor performance with small batch sizes. While BN works well for convolutional networks with large batches, it struggles in scenarios such as recurrent neural networks, online learning, or when computational resources limit batch sizes. Layer Normalization offers a solution by implementing a different normalization strategy that standardizes activations across feature dimensions within individual samples, maintaining complete independence from batch composition. The key advantages of LN over BN include its batch-size independence, making it equally effective whether processing a single sample or thousands simultaneously. This property makes LN particularly valuable for recurrent networks processing sequential data, where batch-level statistics may not be meaningful across different time steps. Additionally, LN eliminates the need to maintain running statistics during inference, simplifying deployment and ensuring consistent behavior regardless of the input batch configuration. These benefits have made LN the preferred normalization technique for transformer architectures and other sequential models where batch-level dependencies would be problematic.

$$\begin{aligned}\mu_l &= \frac{1}{d_l} \sum_{j=1}^{d_l} x_j^{(l)}, & \sigma_l^2 &= \frac{1}{d_l} \sum_{j=1}^{d_l} (x_j^{(l)} - \mu_l)^2 \\ \tilde{x}_j^{(l)} &= \frac{x_j^{(l)} - \mu_l}{\sqrt{\sigma_l^2 + \epsilon}}, & y_j^{(l)} &= \gamma_j \tilde{x}_j^{(l)} + \beta_j\end{aligned}$$

The layer normalization process operates on each individual sample independently and follows these sequential steps:

- μ_l, σ_l^2 : For each sample, LN first computes the mean and variance across all d_l feature dimensions within that specific sample at layer l . Unlike batch normalization which computes statistics across different samples, these statistics are calculated horizontally across the feature vector of a single data point.
- $\tilde{x}_j^{(l)}$: Next, each individual feature $x_j^{(l)}$ is normalized by subtracting the sample’s mean and dividing by its standard deviation. This centers each feature relative to the other features in the same sample and scales them to have unit variance within that sample. The parameter ϵ prevents division by zero when all features have identical values.
- $y_j^{(l)}$: Finally, each normalized feature is transformed using dimension-specific learnable parameters γ_j and β_j . Unlike batch normalization where these parameters are shared across the batch, each feature dimension has its own scaling and shifting parameters, allowing the network to learn optimal transformations for different types of features independently.

This process ensures that for every individual sample, the features are normalized relative to each other, making the normalization completely independent of batch composition and enabling consistent behavior regardless of batch size or the presence of other samples. Research indicates that LN shows particular advantages in scenarios with small batch sizes or sequential data applications, where batch-level statistics can be unreliable. The technique’s batch-independent formulation has proven especially useful for natural language processing and time series modeling. Furthermore, studies suggest that LN provides a more robust option for discriminator networks, as it avoids the batch-dependency issues associated with BN, thereby improving training stability under constrained conditions. It can be applied to both generator and Discriminator

- **Spectral Normalization (SN):** Spectral Normalization represents a fundamentally different approach to stabilizing neural network training compared to both Batch Normalization and Layer Normalization. While BN normalizes activations using batch-level statistics and LN normalizes activations using sample-level feature statistics, SN takes an entirely different route by directly constraining the weight matrices themselves rather than normalizing activations at all. Introduced by Miyato et al. in 2018 [13], the technique works by controlling the spectral properties of the weight matrices, ensuring that each layer’s linear transformation is Lipschitz continuous with a Lipschitz constant of at most 1. This constraint means that the transformation cannot amplify any input signal by more than a factor of 1, effectively controlling how dramatically the output can change relative to changes in the input.

$$\sigma(W) = \max_{\|h\|_2=1} \|Wh\|_2, \quad \bar{W}_{\text{SN}} = \frac{W}{\sigma(W)}$$

The spectral normalization process operates through the following sequential steps:

- $\sigma(W)$: Spectral norm of weight matrix W , defined as the largest singular value of the matrix.

- $\bar{W}_{\text{SN}}$: Spectrally normalized weight matrix, with spectral norm constrained to unity, ensuring 1-Lipschitz continuity.

Implementation Process:

Spectral normalization is applied during each forward pass through the following algorithm:

- **Initialization:** For each weight matrix W in the network, randomly initialize vectors u and v where $u \in \mathbb{R}^{1 \times \text{out_features}}$ and $v \in \mathbb{R}^{\text{in_features} \times 1}$.
- **Power Iteration Method:** Instead of computing the expensive singular value decomposition (SVD) at each step, spectral normalization uses the computationally efficient power iteration method to estimate $\sigma(W)$:

$$\tilde{v} = \frac{W^T u}{\|W^T u\|_2}, \quad \tilde{u} = \frac{W \tilde{v}}{\|W \tilde{v}\|_2}$$

- **Spectral Norm Estimation:** The spectral norm is approximated as:

$$\sigma(W) \approx \tilde{u}^T W \tilde{v}$$

- **Weight Normalization:** The original weight matrix is divided by its estimated spectral norm:

$$\bar{W}_{\text{SN}} = \frac{W}{\sigma(W)}$$

- **Vector Updates:** The vectors u and v are updated with $\tilde{u}$ and $\tilde{v}$ respectively for use in the next iteration. Since weight matrices change slowly during training, only one power iteration per training step is sufficient for accurate estimation.
- **Forward Pass:** The normalized weights $\bar{W}_{\text{SN}}$ are used in place of the original weights W for the current forward pass, ensuring that each layer maintains a Lipschitz constant of at most 1.

This process is typically applied to **all layers in the discriminator** and can optionally be applied to generator layers, though this is less common due to potential restrictions on sample diversity. Based on research it has been established SN as the standard approach for discriminator regularization, particularly in high-resolution image generation applications. Through explicit control of gradient magnitudes, studies have demonstrated that SN significantly reduces mode collapse and improves training stability. While theoretically applicable to generator networks, practical implementations rarely use SN in generators, as excessive constraints have been observed to limit sample diversity and reduce the quality of generated outputs.

2.5 Wasserstein GAN (WGAN) 2017

The Wasserstein GAN was introduced as an improvement to address the training instability of the original vanilla GAN. Problems like **mode collapse** (where the generator produces very limited variety of samples) and **vanishing gradients** are common in vanilla GANs. WGAN tackles these issues by replacing the Jensen-Shannon (JS) divergence with the **Wasserstein distance** (also known as Earth Mover’s distance). The Wasserstein distance measures the cost of transforming one data distribution into another. Unlike JS divergence, it provides a smooth and meaningful gradient even when the real and generated distributions have little to no overlap. This results in a more stable training process. To achieve this, the discriminator is replaced with a **”critic”** that outputs a score rather than a probability. For the math to work correctly, this critic must satisfy a property called **1-Lipschitz continuity**. The original WGAN paper proposed enforcing this constraint through a simple technique called **weight clipping**, where the critic’s weights are clamped to a small, fixed range after each gradient update.

Mathematical Formulation of Objective Function

The WGAN framework changes the objective to approximate the Wasserstein distance, leading to the following minimax game:

$$\min_G \max_{D \in \mathcal{D}} V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)}[D(x)] - \mathbb{E}_{z \sim p_z(z)}[D(G(z))] \quad (2)$$

Where:

- G is the Generator network.
- D is the **Critic** network (no longer a simple discriminator).
- x represents real data samples from the true distribution $p_{\text{data}}(x)$.

- z is random noise sampled from a prior distribution $p_z(z)$.
- $G(z)$ represents generated samples.
- $D(x)$ is the critic’s score for the input x (it’s a real number, not a probability).
- $\mathcal{D}$ represents the set of 1-Lipschitz functions, a constraint enforced on the critic D via **weight clipping**.

Task of Generator

The fundamental objective of the **Generator** is the same as in a traditional GAN: to process a random noise vector and conditional inputs (P, X_0) to synthesize realistic three-dimensional physics outputs $(-\Delta P, \Delta\theta, \Delta r)$. However, instead of trying to fool a binary classifier, the generator’s goal is to produce samples that earn a high “realness” score from the critic. By maximizing this score, the generator implicitly minimizes the **Wasserstein distance** between the real and generated data distributions. This process provides a smoother and more stable gradient, which significantly mitigates the vanishing gradient problem and helps prevent mode collapse.

Task of Critic

The **Critic** replaces the traditional discriminator, and its task is to output an unbounded score that estimates the Wasserstein distance, rather than a probability. It must adhere to a **1-Lipschitz continuity** constraint to function correctly. This constraint is enforced via the hyperparameter **weight clipping** (c): a small value of c can limit the critic’s learning capacity, while a large value can lead to instability. The critic is updated more frequently than the generator, controlled by the hyperparameter n (e.g., $n = 5$ critic updates per generator update), to ensure it provides an accurate gradient. Its architecture is typically an MLP with a final linear activation function to allow for the unconstrained score output.

2.5.1 The Lipschitz Constraint in Wasserstein GANs

A critical requirement of the Wasserstein GAN framework is that the critic function D must belong to the class of 1-Lipschitz functions. This constraint ensures that the critic provides a valid approximation of the Wasserstein distance between the real and generated data distributions. Formally, a function f is 1-Lipschitz if it satisfies:

$$|f(x_1) - f(x_2)| \leq |x_1 - x_2| \quad \forall x_1, x_2. \quad (3)$$

In the WGAN setting, this property guarantees stable gradients and well-behaved optimization dynamics.

Weight Clipping

The original WGAN implementation enforced the Lipschitz constraint through **weight clipping**. After each update of the critic, all of its parameters are restricted to lie within a small fixed interval $[-c, c]$. By bounding the magnitude of the critic’s weights, the function is forced to remain approximately 1-Lipschitz. However, weight clipping introduces certain drawbacks:

- If c is chosen too small, the critic becomes overly constrained and loses its ability to learn complex relationships.
- If c is too large, the constraint is weak, and the critic may violate the Lipschitz condition, reducing stability.
- In practice, weight clipping can lead to capacity underuse and training instabilities.

Spectral Normalization

An alternative method to enforce the Lipschitz constraint is **spectral normalization**. This technique normalizes the weight matrices of the critic by their spectral norm (i.e., the largest singular value), ensuring that each layer has a controlled Lipschitz constant. Compared to weight clipping, spectral normalization has several advantages:

- It provides a more precise and theoretically grounded enforcement of the Lipschitz condition.
- It avoids the capacity loss issues caused by crude weight clipping.
- It has been shown to significantly stabilize GAN training in practice, making it a widely adopted replacement.

In summary, while weight clipping was the original approach to enforce the 1-Lipschitz constraint in WGAN, spectral normalization has emerged as a more robust and effective alternative, allowing the critic to maintain expressive power while still adhering to the theoretical requirements of the framework.

2.6 Training Loop for the Wasserstein GAN

The key distinction of the Wasserstein GAN compared to the standard GAN lies in how the training loop is organized. The critic (formerly the discriminator) and the generator are not updated symmetrically; instead, the critic is trained more frequently to ensure it provides reliable gradient information for the generator. The overall procedure can be described as follows:

1. **Critic Update Phase:** For each generator update, the critic is trained for several consecutive steps (commonly five). In each step, the critic receives two inputs: real samples from the dataset and fake samples produced by the generator from random noise and conditioning variables. The critic computes scores for both sets of samples, and its loss is defined as the difference between the average scores of fake and real samples. After updating the critic, its weights are clipped to lie within a fixed interval $[-c, c]$, enforcing the 1-Lipschitz continuity condition required for approximating the Wasserstein distance.
2. **Generator Update Phase:** Once the critic has been updated for the prescribed number of steps, the generator is updated once. The generator produces new fake samples, which are then scored by the critic. Its objective is to maximize the critic's score for these fake samples, which is equivalent to minimizing the Wasserstein distance between the generated and real data distributions.
3. **Training Dynamics:** This alternating schedule continues for all epochs: several critic updates followed by one generator update. By updating the critic more frequently, the training ensures that it remains a strong and stable guide, while the generator gradually improves its ability to produce realistic samples. Weight clipping plays a central role throughout the loop, as it guarantees that the critic satisfies the Lipschitz constraint needed for the theoretical foundations of the WGAN.

2.7 Conditional Generative Adversarial Networks (cGANs)

A primary limitation of the standard Generative Adversarial Network (GAN) architecture is its inability to control the specific characteristics of the data it generates. The generator learns to map a random noise vector $\mathbf{z}$ to a sample from the data distribution, but there is no mechanism to direct the output. For example, a GAN trained on the MNIST dataset can produce realistic images of handwritten digits, but it cannot be explicitly asked to generate an image of a specific digit, such as a “7”. Conditional Generative Adversarial Networks (cGANs) extend the original GAN framework to address this lack of control by incorporating additional information, known as a condition $\mathbf{y}$, into the model. This conditioning variable $\mathbf{y}$ can be any auxiliary information, such as a class label, a text description, or, in the context of this project, a set of physical parameters.

Architectural Modifications The core innovation of the cGAN is to feed the conditioning information $\mathbf{y}$ to both the generator and the discriminator. This fundamentally changes the task of both networks:

- **Generator:** The generator's input is augmented to include not only the random noise vector $\mathbf{z}$ but also the condition $\mathbf{y}$. Its goal is now to generate a sample

$$\mathbf{x}' = G(\mathbf{z}, \mathbf{y})$$

that is realistic and consistent with the provided condition $\mathbf{y}$.

- **Discriminator:** The discriminator receives both a data sample (either real, $\mathbf{x}$, or fake, $\mathbf{x}'$) and the corresponding condition $\mathbf{y}$. Its task is to determine if the sample is a plausible example *given* that specific condition. It learns a conditional probability

$$D(\mathbf{x}, \mathbf{y}).$$

This architectural change transforms the unconditional generation process into a conditional one, allowing for direct control over the generated output.

Objective Function The min-max game of the cGAN is a straightforward extension of the standard GAN objective. The loss function is conditioned on the variable $\mathbf{y}$, resulting in the following objective function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x}|\mathbf{y})} [\log D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log (1 - D(G(\mathbf{z}|\mathbf{y})))] .$$

In practice, this is implemented by concatenating the condition vector $\mathbf{y}$ with the noise vector $\mathbf{z}$ for the generator's input, and concatenating $\mathbf{y}$ with the data sample $\mathbf{x}$ for the discriminator's input.

Application in muonsimulation Simulation In this project, the cGAN framework is essential for learning the complex, conditional nature of muon propagation through different material.

- The **condition vector** $\mathbf{y}$ consists of the muon’s initial momentum and the material’s radiation length:

$$\mathbf{y} = [P, X_0].$$

- The **generated data** $\mathbf{x}'$ is the set of physical outcomes for a single step:

$$\mathbf{x}' = [-\Delta P, \Delta\theta, \Delta r].$$

By conditioning the GAN on these physical parameters, the model does not learn a single, averaged distribution of interactions. Instead, it learns to approximate the family of probability distributions that describe how interactions vary across different materials and initial energy states, enabling the creation of a versatile and physically realistic simulation tool.

2.8 Training Loop for the Conditional GAN

The training loop for the Conditional GAN closely follows the structure of the standard GAN, with the key distinction that both the generator and discriminator are conditioned on auxiliary information (e.g., physical parameters such as the muon’s initial momentum and the material’s radiation length). The main steps are as follows:

1. **Discriminator Update Phase:** In each iteration, the discriminator receives both real samples paired with their true condition vectors and fake samples produced by the generator given the same conditions. It computes the probability of each input being real. The discriminator’s loss function encourages it to assign high probabilities to real samples with their conditions, and low probabilities to fake samples conditioned on the same inputs.
2. **Generator Update Phase:** The generator is then updated using a fresh batch of noise vectors and corresponding condition vectors. It produces fake samples, which are evaluated by the discriminator. The generator’s objective is to maximize the discriminator’s probability of classifying these conditional fake samples as real. In effect, it learns to generate outputs that are not only realistic but also consistent with the conditioning variables.
3. **Training Dynamics:** The discriminator and generator are updated alternately in each training step, typically one update of the discriminator followed by one update of the generator. By conditioning both networks on the same auxiliary information, the cGAN learns a family of conditional probability distributions rather than a single, unconditional distribution. This allows the model to generate diverse yet physically consistent outcomes for any specified set of conditions.

3 Data Aquisition

The dataset used for this research consists of simulated muon propagation events, obtained from a public repository hosted by CERN. The data was generated using Geant4, a high-fidelity Monte Carlo simulation toolkit widely used in particle physics for modeling the passage of particles through matter. The dataset is structured in the ROOT file format, a standard for high-energy physics data storage and analysis. Each file contains a TTree object, which stores the detailed trajectory of individual muons as they propagate through a given material. For each muon track, the simulation records a series of sequential steps. At each step, key physical variables are logged, including the muon’s 3D position (x, y, z) and 3D momentum (p_x, p_y, p_z) . This granular, step-by-step information provides the ground truth for training the generative models. From these raw kinematic variables, three crucial features describing the muon’s interaction at each step are derived: the momentum loss $(-\Delta P)$, which primarily results from ionization; the scattering angle $(\Delta\theta)$, which is a consequence of multiple Coulomb scattering; and the physical step length (Δr) . These three correlated variables form the core of the simulation task and are the target outputs for the generative models. For training the GAN, only the real dataset containing positive samples is used. Fake samples are not pre-stored or labeled; instead, they are generated dynamically by the generator during training. The discriminator learns to distinguish real data from these generated samples, allowing the model to capture the underlying probability distributions of the muon interaction variables. This approach enables the GAN to serve as an effective complementary tool to Geant4, particularly in applications where computational speed is prioritized over the exhaustive precision of full Monte Carlo simulations.

3.1 Data Normalization and Preprocessing

Data normalization is a critical step in machine learning that ensures features are placed on a comparable scale while preserving their underlying relationships. This process becomes particularly vital for neural networks, where large differences in feature magnitudes can slow convergence or cause training failures [12]. Generative Adversarial Networks (GANs) exhibit heightened sensitivity to scaling issues, as the generator produces synthetic data samples while the discriminator distinguishes between real and generated samples. When feature scales are poorly aligned, training instability emerges, manifesting as exploding or vanishing gradients or mode collapse, where the generator produces only a narrow range of outputs [11]. To address these challenges, this project systematically evaluated a comprehensive range of normalization and transformation strategies for physics data. The following eight transformations were tested: ['MinMax', 'LogMinMax', 'LogStandardMinMax', 'Tanh', 'Sigmoid', 'LinearScale', 'LogTanh', 'LogSigmoid'] These methods encompass both classical scaling techniques and nonlinear mappings (hyperbolic tangent and sigmoid functions), along with log-based variants specifically designed to handle skewed distributions and heavy-tailed variables. While these approaches can improve numerical stability, they may also alter the statistical properties of the data. Since normalization applies mathematical transformations to every observation, it can change the spread, skewness, or overall distribution of the dataset—effects that may impair model assumptions, predictive performance, and comparability across datasets.

3.2 Visual Validation of the Normalization Process

To ensure that the chosen normalization scheme preserves the integrity of the underlying physics, a visual validation strategy was employed. By generating 1D histograms and 2D correlation plots of the data both *before* and *after* the scaling transformations, the impact of the normalization can be qualitatively assessed. This comparison is crucial for verifying that the complex physical relationships between variables are maintained in the transformed space that the GAN operates in.

- **1D Histograms for Marginal Distribution Analysis:** One-dimensional histograms are used to compare the marginal distribution of each physics variable before and after the scaling pipeline is applied. This allows for a direct inspection of how the normalization process—such as a logarithmic transform followed by min-max scaling—reshapes the data. For variables with large dynamic ranges, this visualization confirms that the transformation successfully compresses the distribution into a more uniform and bounded range suitable for the neural network, while highlighting the nature of the statistical alteration.
- **2D Correlation Plots for Physics Relationship Validation:** While 1D histograms show the effect on individual variables, they do not reveal how normalization affects the relationships *between* variables. Two-dimensional correlation plots are essential for this deeper analysis. By comparing these plots before and after scaling, it is possible to verify that the fundamental physical dependencies are preserved in the transformed data space. Key correlations are evaluated to ensure their structure is not lost:
 - *Momentum Loss vs. Initial Momentum* ($-\Delta P$ vs. P): This plot validates that the non-linear relationship is maintained, even after being warped by the scaling functions.
 - *Scattering Angle vs. Initial Momentum* ($\Delta\theta$ vs. P): This visualization confirms that the inverse relationship between scattering and momentum is preserved in the normalized space.
 - *Momentum Loss vs. Scattering Angle* ($-\Delta P$ vs. $\Delta\theta$): This plot ensures that second-order correlations persist through the transformation.
 - *Momentum Loss vs. Step Length* ($-\Delta P$ vs. Δr): This plot inspects the relationship between energy loss and the distance traveled in a single step. It is expected that larger energy loss events correlate with shorter step lengths, particularly in dense materials. Validating this trend ensures the model learns a realistic connection between energy deposition and spatial propagation.

This validation confirms that the data fed to the GAN, while numerically different, still contains the essential physical information required for the model to learn a meaningful and consistent simulation.

3.3 Implementation Framework

To support these transformations reproducibly, a dedicated `Normalizing_Test_GANDataset` class was implemented, building upon and inheriting from the existing `VAEDataloader` class. This modular pipeline extends the established data loading infrastructure while adding GAN-specific functionality to encapsulate the logic for data loading, transformation, and normalization, ensuring consistent preprocessing of all inputs to the GAN. The class returns structured data as a dictionary with specific key organization for efficient training access. The implementation includes the following components:

- **Data Loading and Parsing:** The class uses the `uproot` library to read ROOT files and extract particle trajectory data, including position coordinates, momentum vectors, and energy measurements. File I/O methods iterate through ROOT file directories, extract `TTree` branches containing physics variables (`pos.x`, `pos.y`, `pos.z`, `p.x`, `p.y`, `p.z`, `initial.energy`), and convert the hierarchical ROOT data structure into flat NumPy arrays suitable for tensor operations.
- **Physics Feature Engineering:** Calculates derived physics quantities from raw data, including momentum magnitude, scattering angles using vector geometry, and step lengths from position differences. This is obtained through computational methods that apply physics formulas: momentum magnitude calculation, scattering angles through dot product calculations between consecutive momentum vectors, and step lengths by computing Euclidean distances between successive position coordinates along particle trajectories.
- **Multi-Scale Transformation System:** Implements eight scaling methods (MinMax, LogMinMax, LogStandardMinMax, Tanh, Sigmoid, LinearScale, LogTanh, LogSigmoid) with separate transformation parameters fitted and stored for each material type. This is obtained by creating transformation classes that implement `fit()` and `transform()` methods, where scaling parameters (min/max values, means, standard deviations) are computed during the fitting phase on training data and stored as instance variables, then applied consistently during data retrieval using mathematical transformations that map raw physics values to normalized ranges.
- **Material-Specific Data Organization:** Maintains separate scaling statistics for Lead, Water, Air, and Silicon Dioxide while preserving the relative physics relationships for each material in the normalized space. This is obtained by implementing material-based data partitioning where the class maintains separate dictionaries or data structures for each material type, with independent scaling parameter storage and retrieval methods that ensure cross-material consistency while allowing material-specific normalization characteristics.
- **Efficient Data Access:** Provides methods for retrieving transformed data by material type and transformation method, with in-memory caching to reduce computational overhead during training. This is obtained through implementation of indexing methods, hash-based lookup tables for material and transformation combinations, and memory management strategies that store frequently accessed transformed data in RAM while implementing lazy loading for less common data combinations.
- **Data Quality Check:** Includes validation checks for missing values, outliers, and physically invalid parameter combinations that could affect training stability. This is obtained by implementing validation functions that check for NaN/Inf values, apply statistical outlier detection (using methods like z-score thresholds or interquartile range filtering), and enforce physics constraints (such as ensuring positive energy values, momentum conservation, and reasonable scattering angle ranges) before data is returned to the training pipeline.

The processed physics data is partitioned into two primary components:

- **Conditional tensor:** Contains the conditioning variables that guide generation: the muon momentum P and the material radiation length X_0 . These variables are preserved in their natural physical units and remain fixed during training as non-trainable parameters, providing the context that identifies which material a given sample belongs to and the incoming muon momentum.
- **Target tensor:** Contains the ground-truth variables that the generator must reproduce and the discriminator must classify: momentum loss $-\Delta P$, scattering angle $\Delta\theta$, and step length Δr . A multi-stage normalization strategy (log transform followed by Min-Max scaling) is applied exclusively to these target variables.

The selected pipeline ensures that all target variables fall within a bounded range, improving numerical stability, accelerating convergence, and supporting more reliable adversarial dynamics. This systematic approach to data preprocessing ultimately enhances GAN performance by maintaining the statistical integrity of the physics data while optimizing it for stable neural network training.

3.4 Results and Selection

The empirical outcome of these tests revealed a clear distinction among the methods. Only the log-based Min-Max variants preserved the distributional shape required for reliable adversarial training. Specifically, **Log Min-Max** and **Log Standard Min-Max** produced comparable, well-preserved histogram shapes and supported stable GAN training. All other tested transformations (MinMax, Tanh, Sigmoid, LinearScale, LogTanh, and LogSigmoid) distorted the distributions in the histogram sampling plots, rendering them unsuitable for this physics dataset. These distortions manifested as shifted peaks, excessive compression of dynamic range, or loss of informative tails, which subsequently led to degraded or unstable adversarial training. Given that Log Min-Max and Log Standard Min-Max yielded essentially equivalent outputs in terms of distribution preservation and GAN stability, not only this it also alters the relationship between the different which ca, the simpler and less computationally demanding option—**Log Min-Max**—was selected as the final preprocessing strategy.

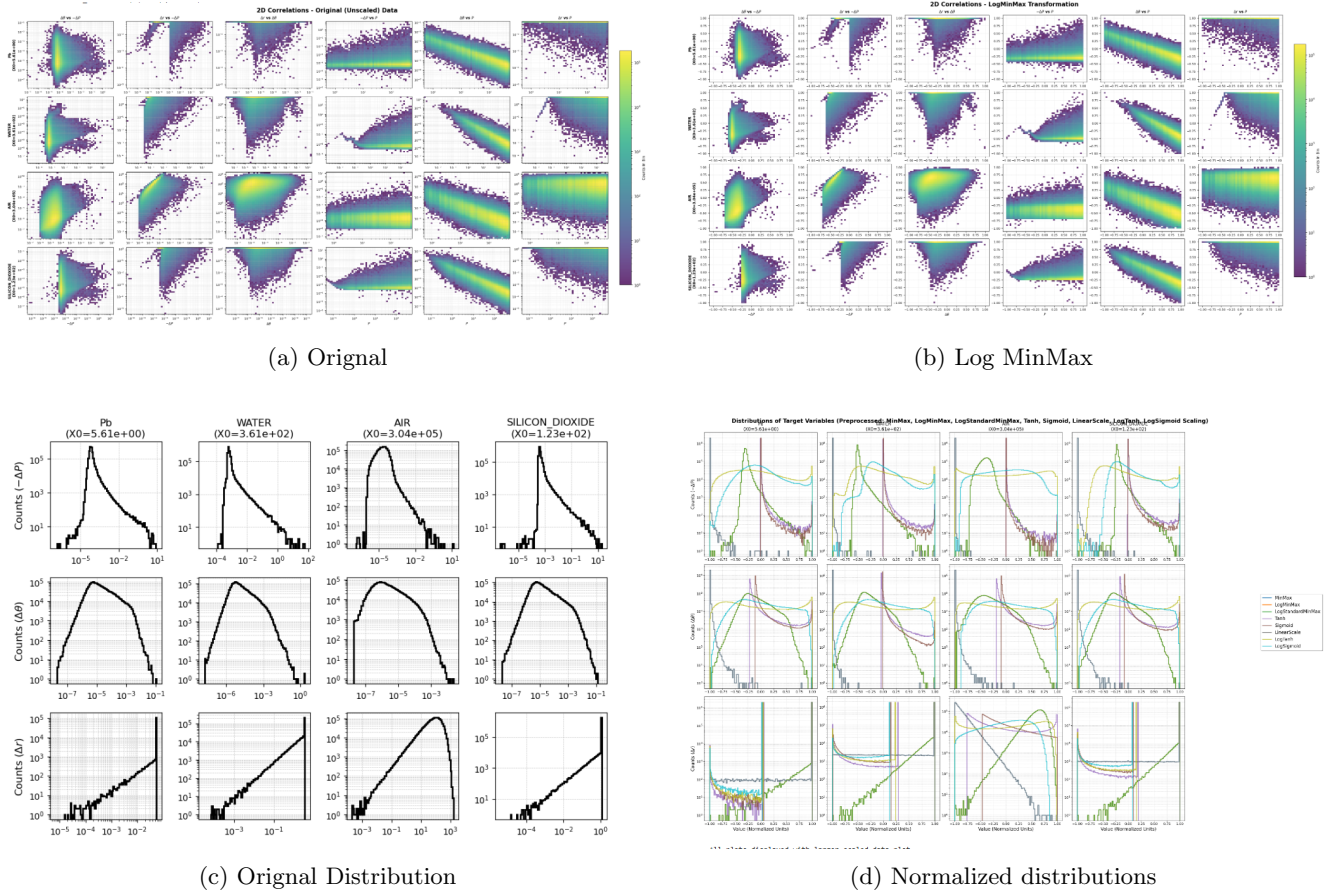


Figure 1: A comparison of distributions for different target variables and preprocessing methods.

4 Code Implementation

Below is an explanation of the code implementation corresponding to the theoretical discussion above. The code described in the following text can be found on GitHub: https://github.com/FlorianBury/MuonPropagation/tree/Shaivya_shankar.

4.1 Modular GAN Class Design.

The Generative Adversarial Network was implemented through a unified PyTorch module that encapsulates both generator and discriminator networks within a single GAN class. This architectural approach provides a modular and flexible framework for adversarial training while maintaining clear separation of concerns between the two competing networks. The class was designed with a comprehensive `build_mlp` method that constructs Multi-Layer Perceptron networks with configurable architectures, allowing both the generator and discriminator to be built using the same unified approach with different input/output specifications. The generator component transforms noise vectors of dimension `noise_dim` into synthetic data of dimension `data_dim`, where the noise vector represents the initial momentum conditions that serve as input to the GAN, effectively imitating the muon simulator's initial parameters. This physics-motivated design allows the generator to learn the mapping from initial momentum states to the corresponding detector responses or particle trajectories, utilizing GELU activation functions in hidden layers without batch normalization or dropout for simplicity. The discriminator processes data samples and outputs a single probability score via a sigmoid activation function, enabling binary classification between real simulated data and GAN-generated synthetic data. The class provides dedicated methods for generation (`generate`), discrimination (`discriminate`), and a flexible `forward` method supporting different operational modes: generator-only sampling, discriminator-only evaluation, or joint adversarial training. This modular design facilitates independent testing of network components while supporting seamless integration during adversarial training phases.

4.1.1 Optimal Discriminator Configuration.

The effectiveness of GAN training critically depends on achieving the right balance between generator and discriminator capabilities. A discriminator that is too weak fails to provide meaningful gradient signals to guide generator improvement, while an overly strong discriminator can dominate the adversarial game, causing generator

gradients to vanish and training to stagnate. This balance is particularly sensitive to the discriminator’s architectural choices—including network depth specified by `discriminator_hidden_dims`, learning rate, and optimizer parameters—as well as training hyperparameters such as the Adam optimizer’s momentum terms. The challenge is compounded by the high-dimensional and non-convex nature of the hyperparameter space, where traditional grid search or random search approaches become computationally prohibitive and often fail to identify optimal configurations. Furthermore, the interdependencies between architectural and training parameters create a complex optimization landscape where the optimal choice for one parameter depends heavily on the values of others, necessitating a more sophisticated search strategy.

Optimal Parameter Selection with Optuna To address these optimization challenges, Optuna was used, an automated hyperparameter optimization framework that leverages the **Tree-structured Parzen Estimator (TPE)** algorithm for intelligent parameter search. Unlike manual trial-and-error approaches, Optuna systematically explores the hyperparameter space using Bayesian optimization principles, significantly reducing the computational cost while improving the likelihood of finding optimal configurations.

TPE Algorithm and Search Strategy. The TPE algorithm operates through a sophisticated probabilistic modeling approach that maintains two complementary Gaussian Mixture Models: $\ell(x)$ representing configurations that yielded superior performance (validation losses in the top 15% of trials), and $g(x)$ representing configurations with poorer performance. The optimization begins with an initial exploration phase (approximately the first 10 trials) during which hyperparameters are sampled uniformly at random from the defined search space to populate these models with performance data. For each subsequent trial, TPE samples candidate configurations from $\ell(x)$, computes the likelihood ratio $\ell(x)/g(x)$, and selects the configuration that maximizes this ratio, effectively focusing search effort on regions associated with historical success while avoiding areas linked to poor performance.

Search Space Definition and Objective Function. The optimization targeted both architectural and training hyperparameters within a carefully defined search space. The learning rate (lr) was selected from $\{10^{-4}, 2 \times 10^{-4}, 3 \times 10^{-3}\}$, the β_1 parameter of the Adam optimizer was sampled uniformly from $[0.5, 0.9]$, network depth (d_{layers}) varied between 2 and 5, and the number of neurons per hidden layer (d_{neurons}) was chosen from $\{64, 128\}$, with hidden dimensionality fixed to d_{neurons} for all layers. A dedicated objective function trained the discriminator for a fixed number of epochs, iterating over batches of real and synthetic data generated from random noise. Binary cross-entropy loss was computed for both real and fake inputs, with validation loss calculated and reported to Optuna via the `trial.report` method after each epoch.

Pruning Mechanisms and Progressive Optimization. Two complementary pruning strategies enhanced computational efficiency during optimization. **Threshold pruning** terminated any trial whose validation loss fell below 0.1, preventing the discriminator from becoming excessively dominant and destabilizing GAN training dynamics. The **Median Pruner** stopped trials whose intermediate validation losses exceeded the epoch-wise median of completed trials, eliminating clearly underperforming configurations early in the training process. As trials accumulated, TPE progressively refined its probabilistic models through distinct phases: early elimination of clearly suboptimal regions (such as learning rates causing training instability), middle-phase convergence toward promising parameter clusters (stable β_1 ranges paired with moderate learning rates), and final fine-tuning within optimal clusters. The entire process remained deterministic through a fixed random seed, ensuring perfect reproducibility of sampling sequences and pruning decisions across experimental runs.

4.2 The GANDataset Class: Data Preparation for Adversarial Training

To support the training of a conditional Generative Adversarial Network (GAN), a data handling class, `GANDataset`, was implemented. Its purpose is to transform raw, simulated physics data into a structured format suitable for conditional, adversarial training. This includes both data organization and a multi-stage normalization procedure.

4.2.1 Inheritance and Core Functionality

The `GANDataset` class inherits from a parent class named `TruthDataset`.

Listing 1: Class definition showing inheritance from `TruthDataset`.

```
class GANDataset(TruthDataset):
    # ... class implementation ...
```

Through this inheritance, `GANDataset` uses the parent class’s logic for data extraction and initial processing. This includes:

1. Reading and parsing `.root` files using the `uproot` and `awkward` libraries.

2. Converting raw position and momentum components into 3D vector objects.
3. Calculating step-wise physical variables: momentum loss (`self.dP`), scattering angle (`self.dtheta`), and step length (`self.dr`).

The main functionality specific to `GANDataset` is implemented in the `process(self)` method, which overrides the empty method in the parent class.

4.2.2 Data Partitioning and Normalization

The `process` method partitions the data into two tensors used for conditional generation:

- **condition tensor:** contains input variables guiding the generator. This tensor includes the muon’s momentum at the start of the step (P) and the material’s radiation length (X_0).
- **target tensor:** contains the output variables that the generator is trained to reproduce, including momentum loss ($-\Delta P$), scattering angle ($\Delta\theta$), and step length (Δr).

Listing 2: Partitioning data into `condition` and `target` tensors within the `process` method.

```
# Inside GANDataset.process()
self.register_object(
    name='condition',
    arrs=[P, X0],
    features=['P', 'X0'],
)
self.register_object(
    name='target',
    arrs=[self.dP, self.dtheta, self.dr],
    features=['-dP', 'dtheta', 'dr'],
)
```

The method also defines a two-stage normalization strategy for both tensors.

Conditional Variable Scaling: The momentum variable P , which has a wide dynamic range, is first log-transformed and then standardized using a `StandardScaler`. The radiation length X_0 is used without scaling, as it is constant for each material.

Listing 3: Normalization pipeline for the `condition` tensor, with scaling applied only to P .

```
# Inside GANDataset.process()
self.register_scaling(
    ScalingStep(names='condition', scaling_dict={'P': logscaler()})
)
self.register_scaling(
    ScalingStep(names='condition', scaling_dict={'P': SklearnScaler(StandardScaler())})
)
```

Target Variable Scaling: The `target` tensor undergoes log-transformation followed by scaling using a `MinMaxScaler` to map values into a bounded interval (e.g., $[-1, 1]$). This ensures the target values are within a consistent range.

4.3 The `GANSamplingCallback` Class: Evaluating Generator Performance

To evaluate the trained generator, a `GANSamplingCallback` class was implemented. This callback provides a standardized way to probe the generator by providing it with a specific set of input conditions and visualizing its generated outputs. This allows for a direct assessment of the model’s ability to learn the conditional probability distributions.

4.3.1 Inheritance and Reusability

Similar to the dataset classes, the callback is designed with an object-oriented approach, inheriting from a parent class named `BaseSamplingCallback`.

Listing 4: Class definition showing inheritance from `BaseSamplingCallback`.

```
class GANSamplingCallback(BaseSamplingCallback):
    # ... class implementation ...
```

This design is highly efficient as it allows the `GANSamplingCallback` to inherit a comprehensive suite of general-purpose methods, including the complex logic for creating 1D, 2D, and pair plots from Matplotlib. The child class only needs to implement the logic that is unique to the GAN.

4.3.2 The process Method: Generating Statistical Samples

The core logic of the callback resides in the `process` method, which is responsible for generating data from the trained model. A key feature of a GAN is that its output is stochastic; for a single input condition, the generator can produce a range of different outputs depending on the random noise vector it receives. To capture this behavior, the `process` method generates `self.N_sample` outputs for each input condition.

Listing 5: The `process` method, which generates multiple samples per condition.

```
def process(self, model):
    # For a GAN, the 'model' is the generator.
    generator = model
    generator.eval()

    # Get the batch of conditions and move to the correct device
    conditions = ...

    # Generate N_sample samples for each condition
    all_samples = []
    for _ in range(self.N_sample):
        # Create a NEW random noise vector for each sample
        noise = torch.randn(conditions.size(0), generator.noise_dim, device=device)
        samples = generator(conditions, noise).cpu()
        all_samples.append(samples.unsqueeze(2))

    # Concatenate along the new sample dimension
    samples = torch.cat(all_samples, dim=2)

    return None, samples
```

As shown in the code, the method loops `self.N_sample` times, creating a new random `noise` vector in each iteration while keeping the input `conditions` the same. This produces a rich statistical dataset that represents the full distribution of the generator's learned output for each specific physical scenario.

4.3.3 The plot_sampling Method: Visualizing the Output

After the `process` method has generated the data, the `plot_sampling` method is called. This method leverages the plotting functions inherited from its parent class.

Listing 6: The `plot_sampling` method, which calls the inherited plotting functions.

```
def plot_sampling(self, y, samples):
    return {
        f'sampling_1D_{self.suffix}': self.plot_1D(samples),
        f'sampling_2D_{self.suffix}': self.plot_2D(samples),
        f'sampling_pair_{self.suffix}': self.plot_pair(samples),
    }
```

It takes the tensor of generated `samples` and passes it to the inherited `plot_1D`, `plot_2D`, and `plot_pair` methods, which then generate a comprehensive set of figures. These figures provide a direct visual assessment of the generator's performance and its ability to correctly model the conditional physics distributions.

Listing 7: Normalization pipeline for the `target` tensor using `MinMaxScaler`.

```
# Inside GANDataset.process()
self.register_scaling(
    ScalingStep(names='target', scaling_dict={'-dP': logscaler(), 'dtheta': logscaler(),
        'dr': logscaler()}))
)
self.register_scaling(
    ScalingStep(names='target', scaling_dict={
        '-dP': SklearnScaler(MinMaxScaler(feature_range=(-1, 1))),
        'dtheta': SklearnScaler(MinMaxScaler(feature_range=(-1, 1))),
        'dr': SklearnScaler(MinMaxScaler(feature_range=(-1, 1)))
    })
)
```

The output of `GANDataset` is a dictionary containing the processed `condition` and `target` tensors, ready to be used with a PyTorch `DataLoader` for batching during GAN training.

4.4 The GANSamplingCallback Class: Evaluating Generator Performance

To evaluate the trained generator, a `GANSamplingCallback` class was implemented. This callback provides a standardized way to probe the generator by providing it with a specific set of input conditions and visualizing its generated outputs. This allows for a direct assessment of the model’s ability to learn the conditional probability distributions.

4.4.1 Inheritance and Reusability

Similar to the dataset classes, the callback is designed with an object-oriented approach, inheriting from a parent class named `BaseSamplingCallback`.

Listing 8: Class definition showing inheritance from `BaseSamplingCallback`.

```
class GANSamplingCallback(BaseSamplingCallback):
    # ... class implementation ...
```

This design is highly efficient, as it allows `GANSamplingCallback` to inherit a comprehensive suite of general-purpose methods, including the complex logic for creating 1D, 2D, and pair plots from Matplotlib. The child class only needs to implement the logic that is unique to the GAN.

4.5 The GANSamplingCallback Class: Evaluating Generator Performance

To evaluate the trained generator, a `GANSamplingCallback` class was implemented. This callback provides a standardized way to probe the generator by providing it with a specific set of input conditions and visualizing its generated outputs. This allows for a direct assessment of the model’s ability to learn the conditional probability distributions.

4.5.1 The BaseSamplingCallback: A Reusable Plotting Engine

The project’s evaluation framework is built upon a general-purpose parent class named `BaseSamplingCallback`. This class is designed to be a reusable plotting engine that can work with any conditional generative model. Its core purpose is to abstract away the complexities of data visualization. It takes a pre-defined set of input conditions (e.g., a range of momentum values and material properties) and contains all the sophisticated, shared logic for generating a comprehensive set of 1D, 2D, and pair plots using Matplotlib. This class acts as a blueprint, providing the full plotting functionality but leaving the model-specific data generation step undefined.

4.5.2 Inheritance and the GAN-Specific Implementation

To create a callback specifically for the GAN, the `GANSamplingCallback` class inherits from `BaseSamplingCallback`.

Listing 9: Class definition showing inheritance from `BaseSamplingCallback`.

```
class GANSamplingCallback(BaseSamplingCallback):
    # ... class implementation ...
```

By inheriting from the base class, `GANSamplingCallback` automatically acquires the complete suite of plotting methods without needing to redefine them. This structure follows the “Don’t Repeat Yourself” (DRY) software engineering principle. The child class only needs to implement the logic that is unique to a GAN—namely, the specific `process` method for generating samples using a generator and a noise vector. All the plotting functionality is automatically handled by the parent class, making the code cleaner, more modular, and easier to maintain.

4.5.3 The process Method: Generating Statistical Samples

The core logic of the callback resides in the `process` method, which is responsible for generating data from the trained model. A key feature of a GAN is that its output is stochastic; for a single input condition, the generator can produce a range of different outputs depending on the random noise vector it receives. To capture this behavior, the `process` method generates `self.N_sample` outputs for each input condition.

Listing 10: The process method, which generates multiple samples per condition.

```
def process(self, model):
    # For a GAN, the 'model' is the generator.
    generator = model
    generator.eval()

    # Get the batch of conditions and move to the correct device
    conditions = ...

    # Generate N_sample samples for each condition
    all_samples = []
    for _ in range(self.N_sample):
        # Create a NEW random noise vector for each sample
        noise = torch.randn(conditions.size(0), generator.noise_dim, device=device)
        samples = generator(conditions, noise).cpu()
        all_samples.append(samples.unsqueeze(2))

    # Concatenate along the new sample dimension
    samples = torch.cat(all_samples, dim=2)

    return None, samples
```

As shown in the code, the method loops `self.N_sample` times, creating a new random `noise` vector in each iteration while keeping the input `conditions` the same. This produces a rich statistical dataset that represents the full distribution of the generator’s learned output for each specific physical scenario.

4.5.4 The plot_sampling Method: Visualizing the Output

After the `process` method has generated the data, the `plot_sampling` method is called. This method is simple, as it leverages the plotting functions inherited from its parent class.

Listing 11: The plot_sampling method, which calls the inherited plotting functions.

```
def plot_sampling(self, y, samples):
    return {
        f'sampling_1D_{self.suffix}': self.plot_1D(samples),
        f'sampling_2D_{self.suffix}': self.plot_2D(samples),
        f'sampling_pair_{self.suffix}': self.plot_pair(samples),
    }
```

It takes the tensor of generated `samples` and passes it to the inherited `plot_1D`, `plot_2D`, and `plot_pair` methods, which then generate a comprehensive set of figures. These figures provide a direct visual assessment of the generator’s performance and its ability to correctly model the conditional physics distributions.

4.6 Visualization of 2D Data Correlations of Original Data

To facilitate a comprehensive analysis of the training data, a function named `plot_2d_correlations_by_transformation` was developed. The primary purpose of this function is to systematically visualize the two-dimensional correlations present in the ground truth Geant4 dataset. It serves as a crucial diagnostic tool for understanding the underlying physics and for validating the effects of each step in the data preprocessing pipeline.

4.6.1 Functionality and Design

The function is designed to be a data exploration tool that operates on the collection of `GANDataset` objects. Its core functionality is to generate a series of grid-based figures that display the 2D joint probability distributions for all relevant pairs of physical variables. The function programmatically determines which correlations to plot, ensuring a complete analysis. This includes two categories of variable pairs:

1. **Target vs. Target:** All unique combinations of the target variables are plotted against each other (e.g., momentum loss $-\Delta P$ vs. scattering angle $\Delta\theta$). This is essential for visualizing the inherent physical correlations that the generative model must learn to reproduce.
2. **Condition vs. Target:** The primary conditional variable, momentum P , is plotted against each of the target variables. This allows for a direct visual inspection of how the output distributions are conditioned on the initial state of the muon.

A key feature of the function is its ability to visualize the data at every stage of the normalization process. It iterates through each transformation defined in the dataset's scaling pipeline (e.g., 'log', 'standard', 'minmax') and generates a separate, dedicated figure for each one, in addition to a figure for the original, unscaled data.

Listing 12: Core loop structure of the plotting function.

```
# Create list with original data first, then all transformations
plot_order = ['Original'] + list(transformations)

# Create separate figure for each data type
for data_type in plot_order:
    print(f"\nGenerating 2D plots for: {data_type}")

    # Setup grid layout
    fig, axs = plt.subplots(...)

    # Loop through materials (rows) and plot types (columns)
    for row_idx, material in enumerate(materials):
        for col_idx, plot_def in enumerate(plot_definitions):
            # ... plotting logic ...
```

4.6.2 Output and Interpretation

The output of the function is a series of figures, each corresponding to a specific stage of data transformation. Within each figure, a grid of 2D histograms is generated, where each row represents a different material and each column represents a different variable pair. This grid layout facilitates a direct comparison of how the 2D correlations differ across materials and how they are affected by the scaling operations. The data density is visualized using a logarithmic color scale (LogNorm) to ensure that low-probability regions of the distribution remain visible. By examining these plots, one can verify that the preprocessing pipeline is functioning as intended and gain a deeper intuition for the complex, high-dimensional probability distributions that the generative models are tasked with learning.

4.7 The plot_distributions Function: Visualizing Data Transformations

A visualization function named `plot_distributions` was developed to support the analysis of the training dataset. This function provides a diagnostic view of the one-dimensional marginal distributions of the target variables, enabling both the inspection of the raw physics data and the validation of the effects introduced by the preprocessing pipeline.

4.7.1 Functionality and Design

The function produces two figures that highlight the progression from raw to preprocessed data:

1. **Original Data Figure:** A compact grid of histograms displaying the target variables in their physical units. Logarithmic axes are used where appropriate to accommodate the wide dynamic ranges and skewed distributions present in the raw data.
2. **Preprocessed Data Figure:** A larger figure that illustrates the normalized data. Each subplot overlays the distributions from successive preprocessing steps (e.g., logarithmic transform, min-max scaling), with distinct colors used to differentiate the transformations. This design provides a direct visual comparison of how each stage modifies the underlying data distribution.

4.8 Hyperparameter Optimization with Optuna

A hyperparameter optimization study was conducted using the Optuna framework [?] to identify a suitable discriminator architecture. The implementation is organized around an objective function, pruning strategies, and the final study execution.

4.8.1 The Objective Function

The optimization procedure is defined through an `objective` function, which manages the training and evaluation of a single set of hyperparameters, referred to as a trial. This function is repeatedly executed by the Optuna study to explore different configurations.

Hyperparameter Sampling. At the start of each trial, the search space is specified using the `trial` object. Sampling functions such as `trial.suggest_categorical` and `trial.suggest_int` generate values for the discriminator’s hyperparameters. These sampled values are then used to instantiate a new model instance for the trial.

Training Environment. In this optimization setting, the discriminator is not trained in conjunction with a generator. Instead, synthetic “fake” data is generated by drawing samples from a standard normal distribution. This provides a controlled baseline for evaluating the discriminator’s learning ability. Reason for doing this is because, when we train GAN generator initially produces random noise, which can be simulated by generating random noise.

Listing 13: Simplified fake data generation within the objective function.

```
# Real data is taken from the data loader
real_targets = batch['target'].to(device)

# Fake data is synthesized from a simple noise distribution
fake_targets = torch.randn(batch_size, target_dim, device=device)

# Discriminator is trained to distinguish between the two
pred_real = discriminator(real_targets, conditions)
pred_fake = discriminator(fake_targets, conditions)
```

Reporting and Pruning. Following each training epoch, validation performance is computed and reported to the Optuna study using the `trial.report()` method. This enables the study to track intermediate results and apply pruning where appropriate.

4.8.2 Pruning Strategies

Two pruning mechanisms are employed to reduce computational cost by terminating unpromising trials early:

1. **Median Pruner:** Configured during study initialization, the **MedianPruner** stops a trial if its intermediate validation loss is worse than the median of previously completed trials at the same epoch.
2. **Custom Threshold Pruner:** Implemented within the validation loop, this manual condition raises an `optuna.exceptions.TrialPruned` exception if the validation loss falls below a threshold of 0.1.

Listing 14: Custom pruning condition in the objective function.

```
# Prune the trial if the discriminator becomes too effective
if epoch_val_loss < 0.1:
    raise optuna.exceptions.TrialPruned()
```

4.8.3 Study Execution

The optimization is initiated with `optuna.create_study`, specifying minimization of validation loss as the objective. The `functools.partial` library is used to provide the required data loaders and parameters to the `objective` function. The `study.optimize()` method then executes the optimization loop over the defined number of trials. After completion, the hyperparameters from the best-performing trial are selected for integration into the final discriminator model.

4.9 Code Implementation of Normalization

The implemented Generator and Discriminator architectures incorporate normalization layers placed between linear transformations and activation functions. Three normalization techniques are applied using PyTorch’s `nn` modules: `nn.BatchNorm1d` for batch normalization, `nn.LayerNorm` for layer normalization, and `spectral_norm` for spectral normalization of weight matrices. This design ensures that activations maintain appropriate statistical properties throughout the forward pass, which is critical for stable GAN training. The modular application of these normalization techniques also allows systematic evaluation of their respective effects on generator quality and discriminator performance in the context of muon propagation modeling.

4.9.1 Code Implementation of the Wasserstein GAN

The implemented Wasserstein GAN (WGAN) architecture closely follows the standard conditional GAN structure, with a generator and a critic network. The generator retains the same design as in the vanilla GAN, combining the condition vector with a noise vector to produce the target outputs. The main architectural difference lies in the critic: unlike the discriminator in the vanilla GAN, the critic does not have an output activation function. Instead, it outputs an unconstrained real-valued score, which is required for approximating the Wasserstein distance. Apart from this modification, the generator and critic architectures are consistent with those used in the standard GAN framework.

4.9.2 Code Implementation of the Wasserstein GAN

The WGAN implementation reuses the generator and discriminator structure of the standard GAN, with the discriminator redefined as a *critic*. Unlike the standard GAN, the critic does not apply an output activation function. Instead of modeling probabilities, the critic outputs real-valued scores, which are used to approximate the Wasserstein-1 distance between the model distribution and the data distribution. To satisfy the 1-Lipschitz constraint required by the Wasserstein objective, two alternative techniques were considered:

- **Weight clipping:** After every critic update, all model parameters are constrained within a fixed interval, typically $[-0.01, 0.01]$. This approach enforces the Lipschitz constraint directly but may limit the capacity of the critic if the clipping value is too small.
- **Spectral normalization:** Applied during model definition by wrapping linear layers with PyTorch’s `spectral_norm` utility. This approach normalizes the spectral norm of each layer’s weight matrix, providing a more stable and theoretically grounded way of enforcing the Lipschitz condition.

Listing 15: Implementing weight clipping and spectral normalization in the WGAN critic.

```
# --- Weight Clipping (applied after every critic update) ---
for p in critic.parameters():
    p.data.clamp_(-clip_value, clip_value)

# --- Spectral Normalization (applied during critic definition) ---
from torch.nn.utils import spectral_norm
self.linear1 = spectral_norm(nn.Linear(in_features, out_features))
```

In practice, the critic is trained more frequently than the generator. The parameter n_{critic} controls this update ratio. For each generator update, the critic is updated n_{critic} times (commonly $n_{\text{critic}} = 5$). This allows the critic to provide reliable gradient estimates before the generator parameters are updated.

5 Results and Discussion

This section presents the results of the experiments that were conducted and provides a detailed discussion of the findings. The subsequent subsections summarize the observed outcomes, analyze their implications, and highlight key insights derived from the experiments.

5.1 Optuna study based Hyperparameters

To identify an optimal architecture for the GAN, two distinct hyperparameter optimization studies were conducted using the Optuna framework. The objective of these studies was to determine the set of hyperparameters—including both the generator and critic architectures, as well as optimizer parameters—that yielded the most stable training dynamics and the highest-quality generator outputs. Importantly, the generator and discriminator were always configured with the same number of layers and neurons in order to maintain architectural symmetry.

The first study was carried out on a dataset consisting of a single material (Lead). The hypothesis was that a relatively low-capacity model (fewer layers and neurons) would be sufficient to learn the statistical properties of this homogeneous data distribution. The optimization confirmed this hypothesis, as the best-performing model utilized only two layers with a moderate number of neurons. The results are summarized below:

- **Best Discriminator:** Final Validation Loss = 0.113380, Trial Number = 4
- **Best Hyperparameters:** `lr = 3e-05`, `beta1 = 0.5682`, `d_layers = 2`, `d_neurons = 128`

The second study was performed on a composite dataset containing four different materials: Air, Lead, Silicon Dioxide, and Water. This dataset introduced greater complexity, requiring the generator to model diverse conditional distributions while the critic distinguished real from fake samples across multiple domains. The hypothesis was that a higher-capacity architecture would be necessary to capture this increased variability. The optimization supported this hypothesis, producing the best results with a deeper and wider model, as summarized below:

- **Best Discriminator:** Final Validation Loss = 0.102380, Trial Number = 7
- **Best Hyperparameters:** $\text{lr} = 3\text{e-}05$, $\text{beta1} = 0.5758$, $\text{d_layers} = 6$, $\text{d_neurons} = 512$

Here, d_neurons denotes the number of neurons per layer in the discriminator, which directly determines the model’s representational capacity. The parameter β_1 corresponds to the exponential decay rate of the first-order moment estimates in the Adam optimizer. It controls how much past gradients influence the current update, thereby affecting the balance between stability and adaptivity of training. In both studies, a value of β_1 between 0.56 and 0.58 was found to be optimal, striking a balance between responsiveness to recent gradients and the smoothing effect of accumulated past gradients. The β_2 parameter, which governs the exponential decay rate of the second-order moment estimates, was left at its default value of 0.999, as it is widely considered robust and stable across GAN training tasks.

Here, d_neurons denotes the number of neurons per layer in the discriminator, which directly determines the model’s representational capacity. The parameter β_1 corresponds to the exponential decay rate of the first-order moment estimates in the Adam optimizer. It controls how much past gradients influence the current update, thereby affecting the balance between stability and adaptivity of training. A value of $\beta_1 = 0.5682$ was found to be optimal in both studies, striking a balance between responsiveness to recent gradients and the smoothing effect of accumulated past gradients. The β_2 parameter, which governs the exponential decay rate of the second-order moment estimates, was left at its default value of 0.999, as it is widely considered robust and stable across GAN training tasks.

5.2 Vanila 2014 GAN Results

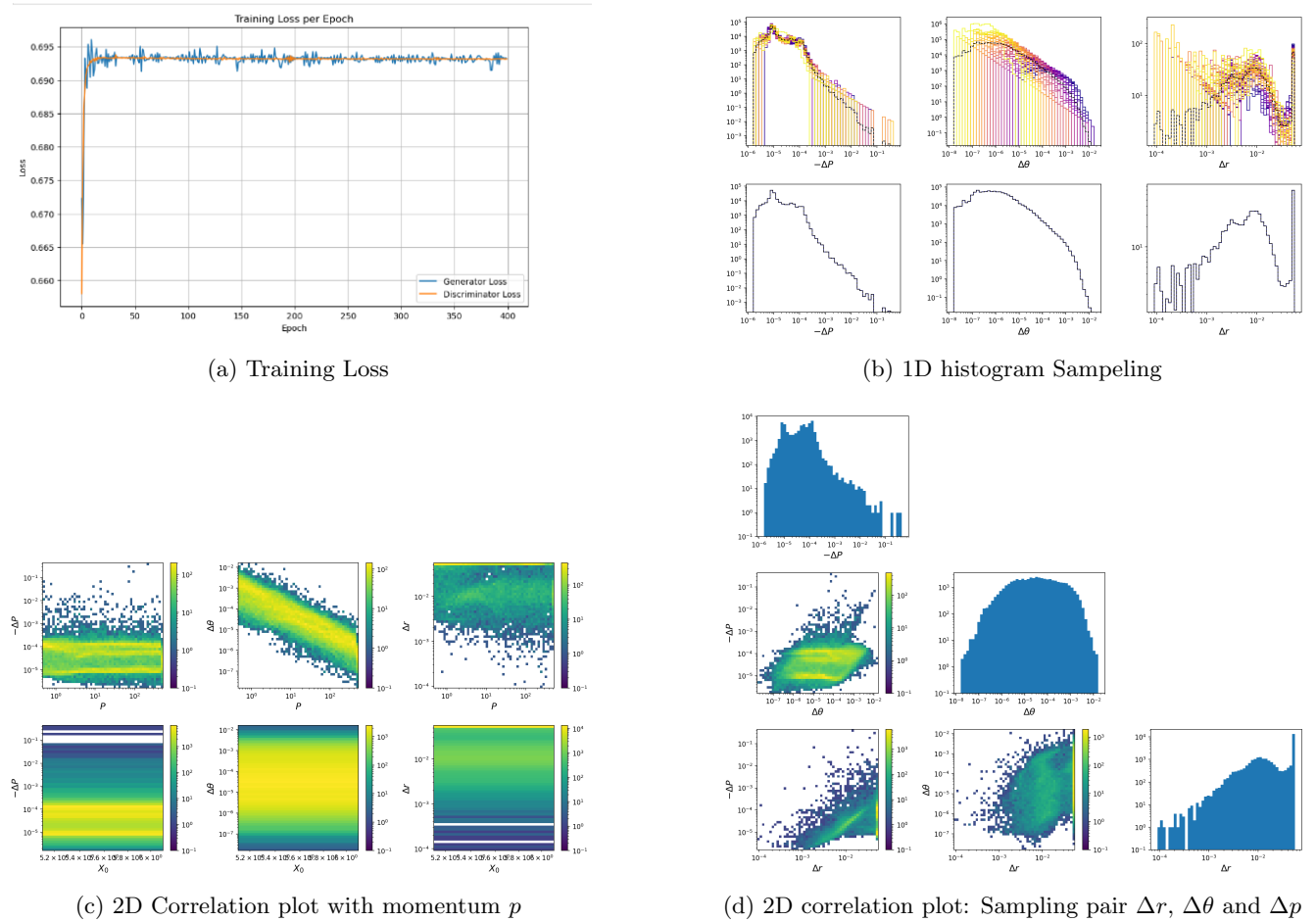


Figure 2: Overview of training results and correlation plots in a 2×2 layout.

The training loss for both the generator and discriminator initially stabilizes around 0.66 and gradually increases to approximately 0.69. This behavior aligns with the theoretical derivation of the GAN objective, which shows that the

global minimum of the virtual training criterion $C(G)$ is achieved when the generator’s distribution matches the data distribution ($p_g = p_{\text{data}}$). At this point, $C(G)$ attains the value $-\log 4$, corresponding to a loss of approximately 0.69 for both networks, as derived from the Jensen–Shannon divergence formulation [4]. The slight increase in both generator and discriminator losses during training indicates that the generator is progressively improving its ability to produce realistic samples, thereby challenging the discriminator more effectively. In a well-balanced training regime, the generator and discriminator reach a dynamic equilibrium where neither network can easily dominate the other. The gradual oscillation around this equilibrium reflects stable adversarial training dynamics. Analysis of the one-dimensional histograms of the generated samples shows that, while the coverage does not perfectly match the original data distribution, the generator successfully captures the main statistical features of the target distribution. Minor discrepancies are expected due to the stochastic nature of GAN training and the finite capacity of the model. Overall, these results indicate that the model has learned a reasonable approximation of the data distribution and exhibits stable adversarial behavior. Since GANs are inherently difficult to train, improvements in the generator can occur gradually and unpredictably. To monitor this, outputs from the generator were saved every 100 epochs. This approach allows observation of when the generator begins producing samples that more closely match the true distribution, reflecting the intermittent and unstable progress characteristic of GAN training.

5.3 Venial Gan with Normalization

5.4 Vanilla GAN with Normalization

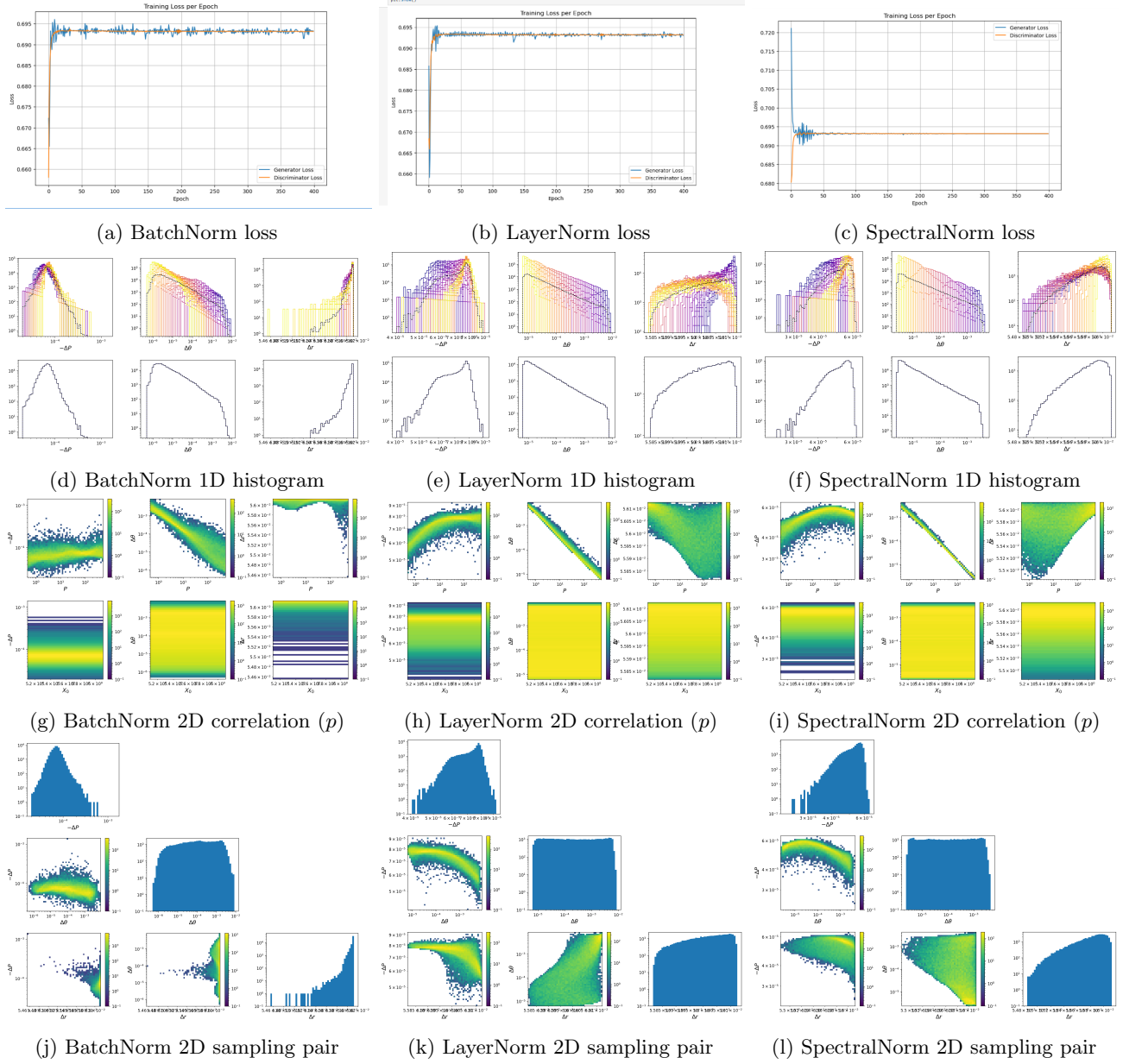


Figure 3: Comparison of normalization techniques (BatchNorm, LayerNorm, SpectralNorm) in GAN training. Each row corresponds to a specific plot type: (1) loss curves, (2) 1D histograms, (3) 2D correlations with momentum p , and (4) 2D sampling pairs.

A comparative analysis of the three normalization methods indicates that Batch Normalization exhibits superior performance in approximating the true distribution of muon propagation. However, all normalization techniques demonstrate reasonably effective behavior. While their performance does not exceed that of the original vanilla GAN, it is plausible that extended training over additional epochs could enhance their effectiveness. Moreover, these methods provide a practical alternative in scenarios where the vanilla GAN exhibits instability. The loss curves of both the generator and discriminator for all the three seems to follow trends similar to those observed in the vanilla GAN, suggesting that normalization contributes to maintaining stable adversarial dynamics without substantially altering training behavior.

5.5 WGAN: Spectral Normalization vs. Weight Clipping

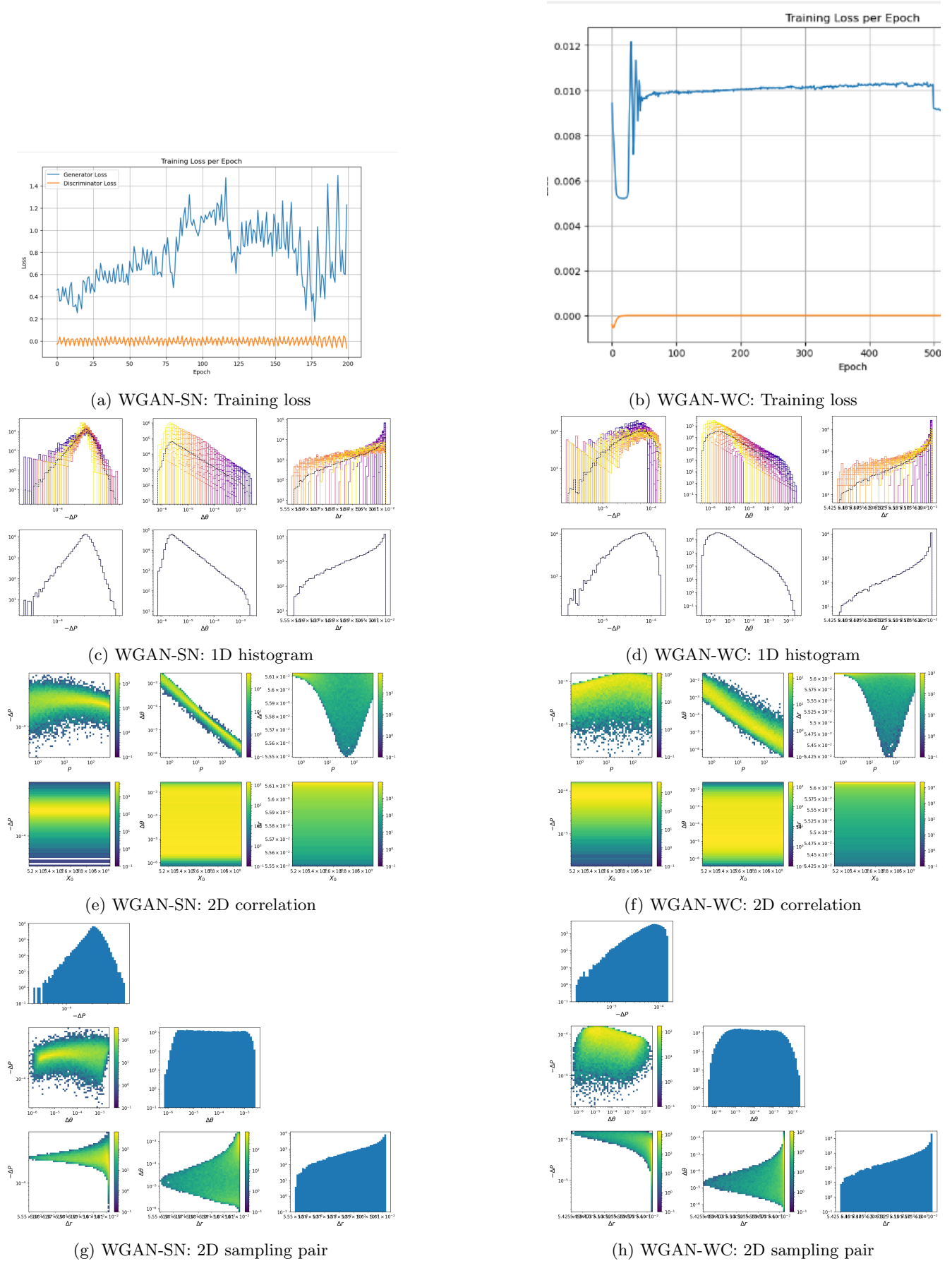


Figure 4: Comparison between WGAN with Spectral Normalization (left column) and WGAN with Weight Clipping (right column). Rows correspond to (1) training loss, (2) 1D histograms, (3) 2D correlations, and (4) 2D sampling pairs.

A comparison of the results shows that both WGAN variants exhibit stable training behavior and generate outputs that approximate the true nature of muon propagation, with good mode coverage. It is possible that training for additional epochs would further improve their ability to match the target distribution. However, WGAN with Spectral Normalization has the practical advantage of requiring fewer hyperparameter settings, making it a more efficient choice. Moreover, prior research has highlighted that weight clipping provides a poor implementation of the 1-Lipschitz constraint, whereas spectral normalization offers a principled improvement. For this reason, spectral normalization is generally preferred over weight clipping. Nevertheless, when compared to the original vanilla GAN implementation, the latter still appears to achieve superior performance under the current experimental setup.

5.6 Conditional GAN: Element-wise Comparison

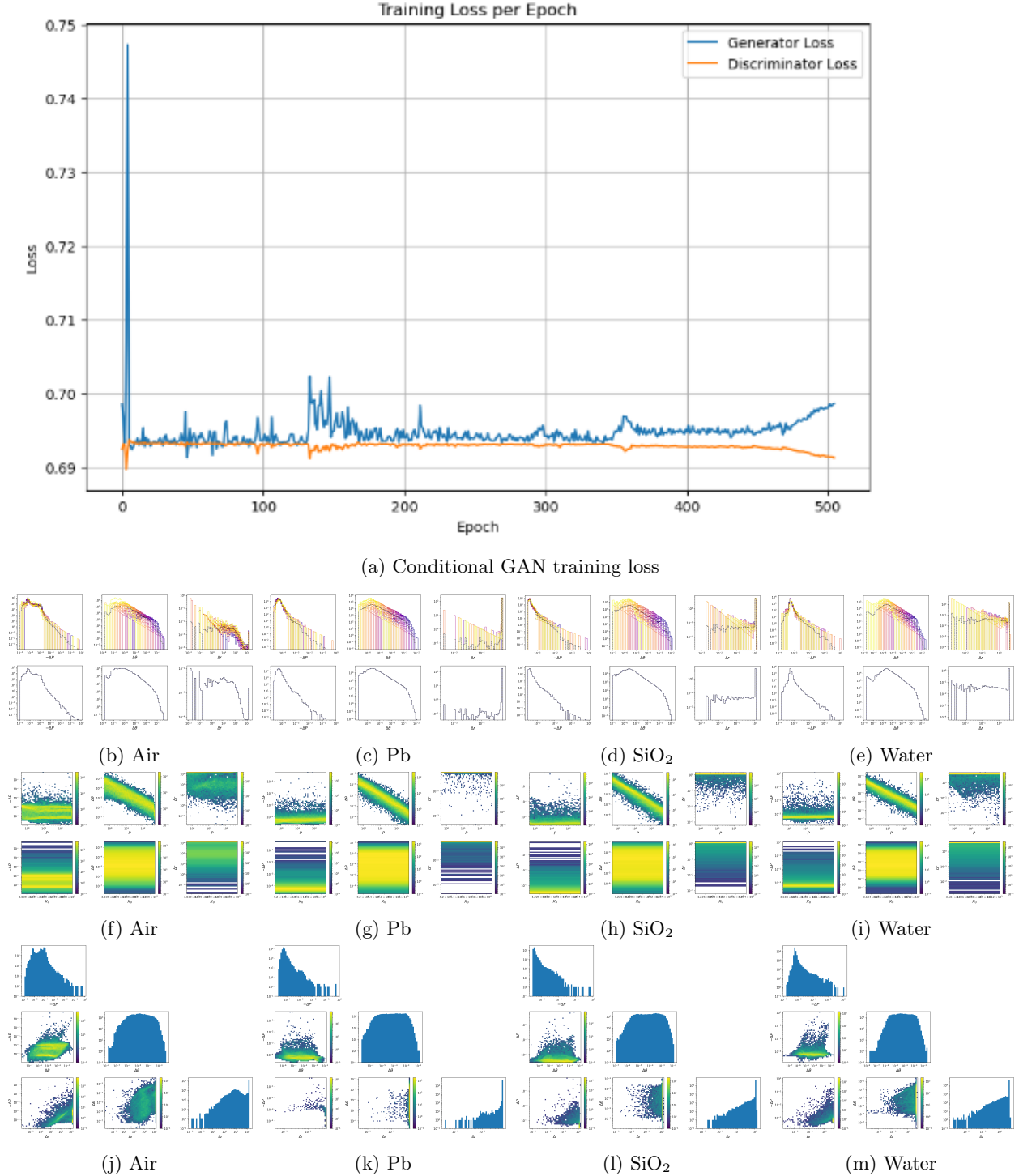


Figure 5: Conditional GAN element-wise comparison. The top row shows the training loss for the GAN. Subsequent rows correspond to 1D histograms, 2D correlations, and 2D sampling pairs for different elements (Air, Pb, SiO₂, Water).

The Conditional GAN demonstrates robust performance across diverse materials, including air, metals, metalloids, and liquids. Although the model does not perfectly replicate the true nature of muon propagation in all cases, it provides promising approximations, indicating that further improvement may be achieved through extended training. Analysis of the training loss shows a pattern consistent with the original GAN: an initially high loss gradually stabilizes around 0.69, with both the generator and discriminator converging to similar values. These observations suggest that the Conditional GAN maintains stable adversarial dynamics while effectively modeling multiple material types.

6 Conclusion and Future Work

6.1 Conclusion

This project successfully investigated the application of Generative Adversarial Networks as computationally efficient surrogates for traditional Monte Carlo simulations in muography. The research systematically progressed from foundational models to more advanced architectures, yielding several key insights. The initial implementation of a **vanilla GAN**, optimized with Optuna, demonstrated that even a basic adversarial framework can learn to approximate the complex, multi-modal distributions of muon interactions, achieving stable training dynamics where generator and discriminator losses converged as theoretically expected. The subsequent evaluation of normalization techniques—Batch, Layer, and Spectral Normalization—confirmed their stabilizing effects but did not yield performance superior to the carefully tuned vanilla GAN, suggesting that for this specific MLP-based architecture, a well-chosen hyperparameter set is paramount. The exploration of **Wasserstein GANs (WGANs)**, using both weight clipping and spectral normalization, reinforced the importance of a stable objective function. While both WGAN variants produced coherent results and good mode coverage, they did not outperform the original GAN in this context and required significant hyperparameter tuning, particularly the weight-clipping version. The most significant advance was achieved with the **conditional GAN (cGAN)**. By conditioning the generation process on physical parameters like initial momentum and material radiation length, the cGAN successfully learned to model the distinct interaction physics across four different materials (Air, Lead, Silicon Dioxide, and Water). This model not only showed stable training but also produced the most physically plausible and versatile results, directly addressing one of the key research questions. In summary, this work validates the potential of GAN-based models to serve as effective, fast simulators in muography. The findings indicate that while classic GANs can be stabilized, conditional architectures are essential for capturing the nuanced, material-dependent nature of particle physics simulations, paving the way for their integration into large-scale scientific workflows.

6.2 Future Work: Full Trajectory Validation

The work presented in this thesis has demonstrated the implementation and training of a conditional Generative Adversarial Network capable of modeling the statistical properties of individual muon interaction steps. Additionally, the framework for extending this single-step generator into a full, multi-step particle propagator, the **GANPropagator**, was developed as part of this project. However, a rigorous, large-scale validation of this propagator was determined to be beyond the scope of the current study. The evaluation presented herein, based on histogram sampling and Cumulative Distribution Functions (CDFs), confirms that the generator has learned to reproduce the marginal and joint probability distributions of the key physical variables ($-\Delta P$, $\Delta\theta$, Δr) on a per-step basis. The next step, which constitutes a research direction for future work, is to evaluate the model’s performance in a more physically rigorous context: the simulation of complete muon trajectories. This requires transitioning from a single-step generator to a full, multi-step particle propagator. This can be accomplished using the **GANPropagator** class, a tool designed for this purpose. The propagator functions by iteratively calling the trained generator in a loop. Starting from a muon’s initial state, it uses the generator to predict the outcome of the first step, updates the muon’s kinematics, and then uses this new state as the input for predicting the subsequent step. This process is repeated until the muon’s energy falls below a defined threshold, resulting in a complete, simulated track.

A validation study would involve two key stages:

1. **Ensemble Simulation:** Generate a large ensemble of muon tracks (e.g., 10^5 or more) using both the **GANPropagator** and the ground truth Geant4 simulation under identical initial conditions.
2. **Quantitative Comparison:** Compare a final, physically relevant observable between the two ensembles. This could be the muon’s final stopping depth, total energy loss, or lateral displacement. The agreement between the distributions of this final observable would be quantified using statistical tests, such as the Kolmogorov-Smirnov test or by generating a coverage plot.

Validating the **GANPropagator** in this manner would move beyond demonstrating statistical similarity and would provide evidence for the model’s cumulative physical accuracy. This represents a test of the GAN’s viability as a complementary instrument for physics simulations and would be a step towards its potential adoption in muography applications.

References

- [1] M. Arjovsky, S. Chintala, and L. Bottou, “Towards principled methods for training generative adversarial networks,” *arXiv preprint arXiv:1701.04862*, 2017.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [3] CERN, “Simulated Muon Propagation Dataset,” *CERN Open Data Portal*, 2025. [Online]. Available: <https://opendata.cern.ch/record/XXXX>
- [4] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 2014, pp. 2672–2680.
- [5] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [6] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [7] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.
- [8] D. P. Kingma and P. Dhariwal, “Glow: Generative flow with invertible 1x1 convolutions,” in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018, pp. 10215–10224.
- [9] K. Kurach, M. Lucic, X. Zhai, M. Michalski, and S. Gelly, “A large-scale study on regularization and normalization in GANs,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019, pp. 3581–3590.
- [10] O. Levy and Y. Goldberg, “Neural word embedding as implicit matrix factorization,” in *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, 2014, pp. 2177–2185.
- [11] Z. Li, Y. Zhang, and H. Wang, “Regularization and normalization techniques in generative adversarial networks: A review,” *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–35, 2022.
- [12] J. Lin, Y. Chen, and X. Liu, “A systematic survey of regularization and normalization in GANs,” in *Advances in Neural Information Processing Systems 34 (NeurIPS)*, 2021, pp. 8934–8946.
- [13] T. Miyato, T. Kataoka, M. Koyama, and Y. Yoshida, “Spectral normalization for generative adversarial networks,” in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018.
- [14] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Technical Report*, 2019.
- [15] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, “Improved techniques for training GANs,” in *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, 2016, pp. 2234–2242.
- [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, pp. 5998–6008.
- [17] Z. Xu, W. Luo, Y. Zhang, and X. He, “A comprehensive survey on generative adversarial networks: Recent advances, challenges, and future directions,” *Neurocomputing*, vol. 408, pp. 234–255, 2020.
- [18] G. Bonomi, P. Checchia, M. D’Errico, et al., “Muon Radiography and Tomography: A Review,” *Instruments*, vol. 4, no. 4, p. 34, 2020.
- [19] S. Agostinelli, et al. (Geant4 Collaboration), “Geant4—a simulation toolkit,” *Nuclear Instruments and Methods in Physics Research Section A*, vol. 506, no. 3, pp. 250–303, 2003.
- [20] L. Oláh, T. G. Bito, G. G. Barnaföldi, et al., “Deep learning for cosmic-ray muon tomography,” *Scientific Reports*, vol. 8, no. 1, p. 1228, 2018.